

Proyecto de referencia desde cero

Carrito web completo con HTML, CSS, JavaScript, DOM y localStorage

Propósito

Construir una aplicación que permita seleccionar productos, validar cantidades, calcular subtotales y total, guardar el pedido y mostrar un recibo. Este estándar se adapta a otras temáticas; no obliga a copiar el carrito.

Estructura mínima

```
mi-proyecto/  
  index.html  
  recibo.html  
  css/styles.css  
  js/products.js  
  js/cart.js  
  js/app.js  
  js/receipt.js  
  README.md
```

Criterios

Elemento	Evidencia esperada
HTML	header, main, section, formularios, labels y salida visible
JavaScript	objetos, arreglos, funciones, condiciones y cálculos
DOM y eventos	addEventListener, renderizado y mensajes
CSS	variables, Grid/Flexbox, responsive y estados
Persistencia	JSON.stringify/JSON.parse con localStorage
Validación	entradas válidas y errores visibles
Integración	entrada ->proceso ->estado ->salida
Git	README y commits pequeños

1. Modelo de datos

```
export const products = [  
  { id: "hamburguesa", name: "Hamburguesa", price: 15000 },  
  { id: "pizza", name: "Pizza personal", price: 18000 },  
  { id: "gaseosa", name: "Gaseosa", price: 5000 }  
];
```

2. HTML principal

```
<header><h1>Carrito de restaurante</h1></header>  
<main>  
  <section><h2>Productos</h2><div id="product-list"></div></section>  
  <section>  
    <h2>Pedido</h2>  
    <div id="cart-list"></div>  
    <p>Total: <strong id="cart-total">$0</strong></p>  
    <p id="form-message" role="status"></p>  
    <button id="checkout-button" type="button">Finalizar</button>  
  </section>  
</main>
```

3. Funciones

```
const cart = [];  
function addToCart(productId, quantity) {  
  const amount = Number(quantity);  
  if (!Number.isInteger(amount) || amount <= 0) {  
    throw new Error("La cantidad debe ser un entero positivo");  
  }  
  const product = products.find(item => item.id === productId);  
  if (!product) throw new Error("Producto no encontrado");  
  const existing = cart.find(item => item.id === productId);  
  if (existing) existing.quantity += amount;  
  else cart.push({ ...product, quantity: amount });  
}  
function calculateTotal() {  
  return cart.reduce((sum, item) => sum + item.price * item.quantity, 0);  
}
```

4. DOM y eventos

```
function renderCart() {
  cartList.innerHTML = "";
  for (const item of cart) {
    const row = document.createElement("article");
    row.textContent = `${item.name} - ${item.quantity * item.price}`;
    cartList.append(row);
  }
  totalOutput.textContent = `$$${calculateTotal()}$`;
}
productList.addEventListener("click", event => {
  const button = event.target.closest("[data-add-product]");
  if (!button) return;
  try { addToCart(button.dataset.addProduct, 1); renderCart(); }
  catch (error) { message.textContent = error.message; }
});
```

5. Persistencia y recibo

```
localStorage.setItem("restaurant-cart", JSON.stringify(cart));
window.location.href = "recibo.html";

const saved = JSON.parse(localStorage.getItem("restaurant-cart") || "[]");
```

6. CSS

```
:root { --ink:#172033; --surface:#fff; --canvas:#f3f6f9;
  --primary:#1f5f96; --line:#d6e0e8; --radius:12px; }
.product-grid { display:grid;
  grid-template-columns:repeat(auto-fit,minmax(210px,1fr)); gap:1rem; }
button:focus-visible { outline:3px solid #78aeda; }
button:disabled { opacity:.55; cursor:not-allowed; }
@media (max-width:650px) { main { padding:.75rem; } }
```

7. README y commits

```
# Nombre del proyecto
## Integrantes y roles
## Objetivo
## Como ejecutar
## Flujo principal
## Validaciones
## Estado actual y pendientes
```

```
feat: renderiza catalogo de productos
feat: agrega productos y calcula total
fix: valida cantidades enteras positivas
feat: guarda datos y genera recibo
style: crea componentes y responsive
docs: documenta integrantes y ejecucion
```

8. Pruebas

1. Abrir sin errores 404.
2. Probar entrada válida y errores.
3. Confirmar cálculos y actualización del DOM.
4. Recargar y verificar persistencia.
5. Revisar móvil, teclado, foco visible y mensajes.
6. Confirmar integrantes, roles y autoría.

Meta del referente

Jerónimo Rodríguez Peña debe cerrar persistencia y recibo, añadir validaciones explícitas, responsive, variables CSS y documentación. Ser referente significa avanzar hacia un flujo completo y explicable, no conservar el mismo puntaje.